DIRETRIZES DE REPOSITÓRIO E FLUXO DE DESENVOLVIMENTO

Projeto Horizonte | Guia de contribuição para Backend, Frontend e Inteligência Artificial

Guia de contribuição para as equipes de Backend, Frontend e Inteligência Artificial.

Regra principal: cada tarefa deve ter sua própria Issue e branch. Nenhum aluno deve enviar alterações diretamente para `main`: toda integração deve ocorrer por **Pull Request (PR)**, após revisão.

1. Estrutura do repositório

O Projeto Horizonte utiliza um **monorepo modular**: um único repositório GitHub, `projeto-horizonte`, com diretórios separados para cada responsabilidade.

```
projeto-horizonte/
+-- backend/            # APIs, autenticação, persistência e regras de negócio
+-- frontend/           # Interface, navegação, dashboards e gamificação
+-- ai/                 # Agentes, RAG, LLMs e processamento de IA
+-- contracts/          # Contratos de comunicação e schemas compartilhados
+-- docs/               # Arquitetura, instruções e decisões técnicas
+-- infra/              # Docker e configurações de ambiente
+-- .github/            # Templates de Issues e Pull Requests
+-- README.md           # Visão geral e início rápido
+-- CONTRIBUTING.md     # Regras de colaboração
+-- compose.yaml        # Execução integrada, quando disponível
+-- .env.example        # Exemplos de variáveis sem valores secretos
+-- .gitignore          # Arquivos que não devem ser versionados
```

Pasta	Responsabilidade	Orientação
`backend/`	API, dados, autenticação e regras de negócio	A equipe altera seu módulo e respeita contratos publicados.
`frontend/`	Páginas, componentes e experiência do usuário	Pode usar dados simulados até que a API esteja disponível.
`ai/`	Agentes, RAG, modelos, inferência e avaliações	Pode criar simulações dos serviços dependentes.
`contracts/`	Endpoints, formatos de requisição/resposta e schemas	Mudanças exigem alinhamento entre equipes afetadas.
`docs/`	Documentação técnica e decisões	Deve acompanhar mudanças relevantes de arquitetura.
`infra/`	Configuração local e integração dos serviços	Mudanças que afetam outras equipes precisam ser comunicadas.

Importante: ter pastas separadas não significa que os serviços estejam automaticamente integrados. As equipes devem combinar antes os contratos de API, os nomes de campos, os códigos de erro e a forma de autenticação. Não

incluir senhas, tokens, chaves de API ou arquivos `.env` reais no Git.

2. Estratégia de branches

Será utilizado **GitHub Flow** com uma única branch compartilhada: `main`. Para cada Issue implementável, crie uma **branch temporária** derivada da versão atual da `main`.

Formato: `tipo/area-numero-descricao-curta`

```
main
+-- feat/backend-12-login
+-- feat/frontend-15-tela-login
+-- feat/ai-18-consulta-rag
+-- fix/backend-21-validacao-token
+-- docs/arquitetura
+-- chore/estrutura-inicial
```

Tipos sugeridos: `feat` (funcionalidade), `fix` (correção), `docs` (documentação), `chore` (organização/configuração) e `refactor` (reestruturação sem alterar comportamento).

Regras para todos os colaboradores

- **Não fazer push diretamente na `main`.**
- **Uma Issue por entrega verificável; uma branch por Issue.** Para documentação ou manutenção sem Issue prévia, abrir uma Issue antes de desenvolver.
- Não usar branches permanentes `backend`, `frontend` ou `ai`.
- Não misturar funcionalidades independentes no mesmo PR.
- Fazer commits pequenos, com mensagens que descrevam a alteração.
- Abrir PR para `main`, solicitar revisão e corrigir observações antes do merge.
- Executar os testes disponíveis **localmente** e informar o resultado no PR.
- Após aprovação, usar **Squash and Merge** e excluir a branch temporária.

3. Fluxo de trabalho do aluno: da Issue à entrega

Passo 1 — Escolher e compreender a Issue

No GitHub Project já existente, selecione uma Issue atribuída a você. Leia o objetivo, os critérios de aceitação, as dependências e os módulos envolvidos. Se algo estiver indefinido, registre a dúvida nos comentários **antes de começar**. Atualize o andamento da tarefa no Project.

Passo 2 — Criar sua branch a partir de `main`

No terminal, dentro do repositório local:

```
git switch main
git pull origin main
git switch -c feat/backend-12-login
```

Troque `backend-12-login` pelo nome correspondente à **sua** tarefa. Não crie a branch a partir de uma versão desatualizada de `main`.

Passo 3 — Implementar e testar localmente

Altere apenas os arquivos necessários. Siga os contratos compartilhados e atualize a documentação quando necessário. Antes de registrar o trabalho:

1. Execute a aplicação ou o módulo modificado.
2. Execute os testes disponíveis e verifique o comportamento esperado.
3. Revise os arquivos alterados com `git status` e `git diff`.
4. Confirme que não há credenciais ou arquivos gerados indevidos no commit.

Passo 4 — Fazer commit e publicar a branch

```
git status
git add backend/
git commit -m "feat(backend): implementa login"
git push -u origin feat/backend-12-login
```

O comando `git add backend/` é **apenas um exemplo**. Inclua somente os arquivos da sua tarefa; se você alterou `contracts/` ou `docs/`, adicione os arquivos correspondentes explicitamente.

Passo 5 — Abrir um Pull Request (PR)

No GitHub, clique em **Compare & pull request** ou vá em **Pull requests → New pull request**. Defina:

- **Base:** `main`.
- **Compare:** sua branch de trabalho.
- **Título:** objetivo da mudança (ex.: `feat(backend): login de usuários`).
- **Descrição:** o que foi desenvolvido, como testar, resultados dos testes e observações relevantes.
- **Vínculo com a Issue:** inclua `Closes #12` na descrição (substitua pelo número correto).
- **Revisor:** solicite revisão a outro membro autorizado ou ao responsável técnico.

Não considere um PR aberto como tarefa finalizada. Ele ainda precisa ser analisado e integrado.

Passo 6 — Revisão, correções e integração

O revisor verifica legibilidade, aderência à Issue, contratos, testes locais e ausência de dados sensíveis. Se solicitar mudanças, corrija na **mesma branch** e faça novo `git push`: o PR será atualizado automaticamente.

Depois da aprovação, um colaborador autorizado realiza **Squash and Merge** para `main`. Confira o fechamento da Issue vinculada e exclua a branch de trabalho no GitHub.

Passo 7 — Atualizar seu repositório local

```
git switch main
git pull origin main
git branch -d feat/backend-12-login
```

Se a branch local ainda não puder ser excluída porque o merge foi feito por squash, confirme primeiro que o PR foi integrado; só então remova a branch local com `git branch -D nome-da-branch`.

Resumo do fluxo: `Issue → branch → implementação → testes locais → commit/push → Pull Request → revisão → Squash and Merge → Issue concluída`.

4. Documentação obrigatória do repositório

A documentação é parte da entrega e deve ser mantida junto ao código.

Documento	O que deve conter
`README.md`	Objetivo do Horizonte; módulos; requisitos básicos; links para a documentação; como iniciar o projeto.
`CONTRIBUTING.md`	Convenção de branches e commits; como trabalhar com Issues; passos de PR, revisão e integração.
`docs/ARCHITECTURE.md`	Responsabilidades dos módulos; fluxos de comunicação; limites e dependências; decisões arquiteturais.
`docs/DEVELOPMENT.md`	Pré-requisitos; configuração local; variáveis de ambiente; comandos de execução e testes; solução de problemas comuns.
`contracts/`	Contratos de interface entre módulos, como OpenAPI e JSON Schema, quando aplicáveis.

Ao alterar comportamento, API, configuração ou estrutura, atualize os documentos afetados no mesmo PR. Não é necessário inventar tecnologias ou comandos de execução antes de a equipe defini-los.

5. Critérios mínimos para considerar uma entrega concluída

[ ] A Issue tem objetivo e critérios de aceitação atendidos.

[ ] A implementação foi feita em branch própria, não diretamente na `main`.

[ ] Foram executadas verificações/testes locais relevantes, com resultados descritos no PR.

[ ] O código não contém segredos, credenciais nem artefatos indevidos.

[ ] Contratos e documentação foram atualizados, quando aplicável.

[ ] O Pull Request referencia a Issue correta e foi revisado.

[ ] O PR foi integrado na `main` e a Issue foi concluída no GitHub Project.

Diretriz do Horizonte: um repositório, três frentes de desenvolvimento, uma branch por tarefa e integração via Pull Request revisado. O projeto **não utiliza GitHub Actions nem CI automatizada** nesta etapa: a responsabilidade pelos testes e pela apresentação de evidências é dos colaboradores e revisores.